

Capa de datos para un copiloto RAG interno de una distribuidora mayorista

Martín Madrid César Andrés Orellana Nicolás Valentín Ciarrapico

Trabajo Práctico Integrador – Bases de Datos para Inteligencia Artificial
Carrera de Especialización en Inteligencia Artificial – FIUBA
2026

Resumen

Se diseña la capa de datos de un copiloto RAG interno para una distribuidora mayorista. La información está repartida entre documentos, procedimientos, condiciones comerciales y datos operativos. Por eso, encontrar una respuesta confiable puede llevar tiempo y depender del conocimiento informal de otras personas.

Se usa PostgreSQL como motor principal, con las extensiones `pgvector` y `btree_gist`. La misma base guarda datos operativos, metadatos documentales, fragmentos, embeddings, permisos y auditoría. Así se aplican la vigencia y los permisos antes de buscar por similitud, y las fuentes vencidas o no autorizadas quedan fuera del ranking.

La implementación incluye datos sintéticos reproducibles, siete consultas funcionales y dos pruebas de seguridad y auditoría. Las restricciones, las políticas RLS (seguridad a nivel de fila) y la evidencia de cada respuesta se validan mediante dos cargas limpias equivalentes. La prueba termina en la capa de datos: no incluye interfaz, API ni un modelo de lenguaje real.

Índice

1. Descripción del caso de uso	3
2. Relevamiento de datos necesarios	3
3. Clasificación de los datos según su tipo	4
4. Modelo conceptual	5
4.1. Operación comercial y logística	5
4.2. Documentos y recuperación	7
4.3. Acceso, interacción y evidencia	9
5. Modelo de implementación según la tecnología elegida	9
5.1. Modelo lógico relacional	9
5.2. Modelo físico en PostgreSQL	13
6. Decisiones de normalización, embebido, referencia y desnormalización	17
7. Justificación de la tecnología seleccionada	17
8. Implementación mínima realizada	18
9. Datos de ejemplo utilizados	18
10.Consultas representativas	19
11.Propuesta para datos semiestructurados, no estructurados y vectoriales	19
12.Propuesta de arquitectura de datos	20
13.Estrategia de seguridad, permisos y aislamiento	22
14.Consideraciones de escalabilidad y rendimiento	22
15.Conclusiones	23
A. Reproducibilidad y material de apoyo	23

1. Descripción del caso de uso

Se eligió un sistema RAG para consultar documentación técnica y se lo adaptó a una distribuidora mayorista. La información está repartida entre documentos y datos operativos. Encontrar una respuesta puede llevar tiempo, depender de alguien que conozca el proceso o terminar en el uso de una versión que ya no está vigente.

Se propone la capa de datos que permitiría a un copiloto interno recuperar información y usarla como fuente para responder. El alcance incluye almacenamiento, integridad, autorización, recuperación y trazabilidad. El entrenamiento de modelos y la aplicación completa quedan fuera de la prueba.

Se consideran tres perfiles. **Operaciones y Logística** consulta fichas de producto, procedimientos e incidencias autorizadas. **Comercial y Compras** consulta condiciones comerciales, políticas de venta e información de proveedores. **Administración y Calidad** accede a documentación legal y de cumplimiento, y revisa la trazabilidad. Los servicios de ingesta, consulta y administración documental no son usuarios finales y reciben los permisos mínimos para su tarea.

La base registra versiones documentales, identifica cuál está vigente, divide su contenido en fragmentos y asocia embeddings. También resuelve consultas estructuradas sobre pedidos, entregas o condiciones comerciales. Cada respuesta exitosa guarda la evidencia que identifica la fuente utilizada.

Los riesgos principales son recuperar contenido no autorizado, usar una versión sustituida o revocada y responder sin una fuente verificable. Para evitarlos, se conserva el historial, se aplican los permisos antes de la recuperación y se guarda evidencia y auditoría de cada interacción.

El alcance se limita a la capa de datos. No se incluyen frontend, backend, API, autenticación, integración con un LLM ni embeddings generados por un modelo real. Esos componentes podrían consumir la base en una etapa futura.

2. Relevamiento de datos necesarios

Los datos se separan según su función. Un pedido se consulta con filtros y relaciones exactas; un procedimiento escrito se recupera por similitud semántica. Cada grupo también tiene un ciclo de vida distinto.

Grupo	Datos principales	Uso dentro de la solución
Operativos	Productos, categorías, clientes, segmentos, proveedores, pedidos, líneas de pedido, entregas e incidencias.	Responder consultas exactas del negocio y relacionar documentos con el contexto operativo.
Comerciales	Condiciones comerciales, precios, descuentos y períodos de vigencia.	Consultar qué condición corresponde a un cliente y producto en una fecha determinada.
Documentales	Documentos, versiones, clases documentales, sensibilidad, archivos y fragmentos.	Mantener el corpus, su procedencia, su vigencia y la unidad de recuperación.
Vectoriales	Modelos de embedding y vectores de cada fragmento.	Resolver búsquedas por similitud sobre documentación autorizada.
Acceso	Actores, perfiles autorizados y permisos por clase documental.	Limitar qué documentos puede recuperar cada perfil.
Interacción y auditoría	Consultas, respuestas, evidencias y eventos de auditoría.	Explicar respuestas anteriores y registrar acciones relevantes.

Cuadro 1: Datos relevados para la solución.

Las preguntas con una respuesta exacta no se resuelven por similitud. La condición comercial

vigente se obtiene por cliente, producto y fecha. Los pedidos demorados se obtienen al relacionar pedidos, entregas e incidencias. La estructura permite aplicar restricciones, agregar resultados y evitar ambigüedades.

El corpus requiere más que el texto. Cada documento tiene una clase, sensibilidad, procedencia y una o varias versiones. La versión conserva la ruta relativa, el nombre, el tipo MIME, el tamaño y el checksum SHA-256 del archivo. Así se verifica su identidad sin guardar el contenido completo en la base.

Cada versión se divide en fragmentos. El fragmento es la unidad vectorizada y la que se muestra como evidencia. Así se identifica la sección y la versión que sostuvieron una respuesta, en lugar de señalar el documento completo.

Cada perfil puede ver distintas clases documentales. El permiso relaciona un perfil con una clase; si la relación no existe, el acceso se deniega. La interacción y la auditoría registran quién consultó, con qué perfil, qué obtuvo y qué evidencia se usó, sin copiar contenido sensible.

Se usan datos sintéticos para no exponer información personal, comercial ni documentos internos. La misma carga se puede repetir con iguales conteos, archivos, vectores y resultados esperados.

3. Clasificación de los datos según su tipo

La clasificación considera forma, función y sensibilidad. Una entidad puede pertenecer a más de una categoría: una consulta es un dato operativo y también puede alimentar análisis posteriores.

Tipo	Ejemplos en la solución
Estructurados	Catálogos, productos, clientes, proveedores, condiciones comerciales, pedidos, entregas, permisos, consultas y evidencias. Se representan con columnas tipadas, claves y restricciones.
Semiestructurados	Metadatos de versiones, parámetros y contenido de resultados estructurados, y detalles de auditoría. Se guardan en JSONB porque su forma puede variar dentro de límites controlados.
No estructurados	Archivos Markdown de las versiones documentales. Permanecen fuera de PostgreSQL y se vinculan mediante ruta, tipo MIME, tamaño y checksum.
Vectoriales	Embeddings sintéticos de los fragmentos documentales, almacenados como VECTOR(32) y asociados a una versión identificable del modelo.
Operativos	Pedidos, líneas, condiciones comerciales, entregas, incidencias, consultas y respuestas. Sostienen la operación o registran una interacción concreta.
Analíticos	Totales por categoría, rankings por segmento, métricas de uso y planes de ejecución. En la prueba se calculan bajo demanda y no justifican un Data Warehouse separado.
Sensibles o restringidos	Condiciones comerciales, incidencias y documentos legales o de cumplimiento. Su visibilidad depende del perfil y de la clase documental.
Auditoría y trazabilidad	Eventos de consulta, acceso, publicación, recuperación y evidencia. Conservan referencias y contexto, pero no duplican el contenido sensible.

Cuadro 2: Clasificación de los datos de la solución.

Los datos analíticos no se guardan por separado porque el volumen y las consultas del TP no lo justifican. Si el historial de uso o la auditoría afectaran al núcleo transaccional, podrían replicarse en otra capa sin cambiar el modelo operativo.

4. Modelo conceptual

La Figura 1 resume las tres áreas del dominio. Las figuras siguientes muestran cada una por separado para que las cardinalidades se lean con claridad.

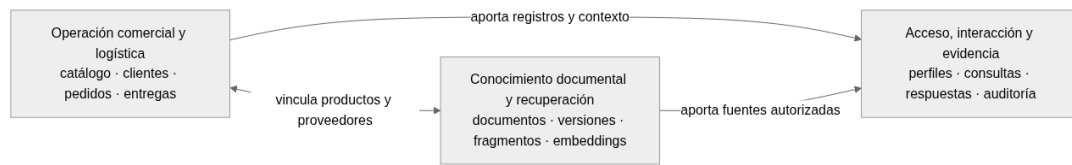


Figura 1: Mapa conceptual general.

4.1. Operación comercial y logística

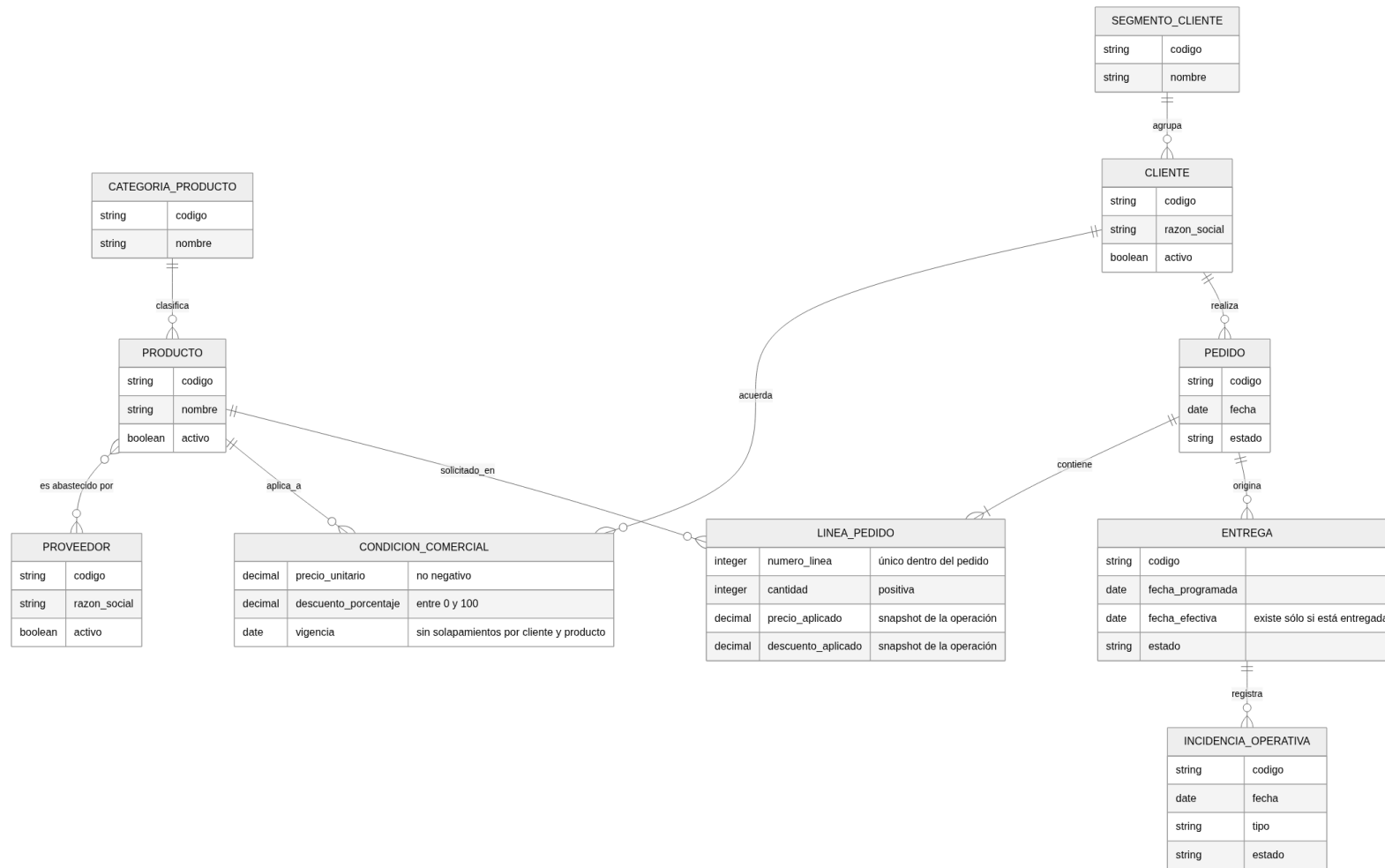


Figura 2: Vista conceptual de la operación comercial y logística.

La Figura 2 relaciona clientes, productos, condiciones comerciales, pedidos y entregas. Un pedido pertenece a un cliente y contiene una o más líneas. Puede tener entregas parciales, y cada incidencia se registra sobre una entrega. Las condiciones comerciales conservan su vigencia; las líneas del pedido guardan el precio y el descuento aplicados en el momento de la operación.

4.2. Documentos y recuperación

La Figura 3 muestra el recorrido desde un documento hasta sus versiones, fragmentos y embeddings. Una versión puede permanecer como borrador durante la carga. Sólo las versiones publicadas, vigentes y autorizadas participan de la recuperación. Las versiones sustituidas o revocadas se conservan para explicar respuestas anteriores.

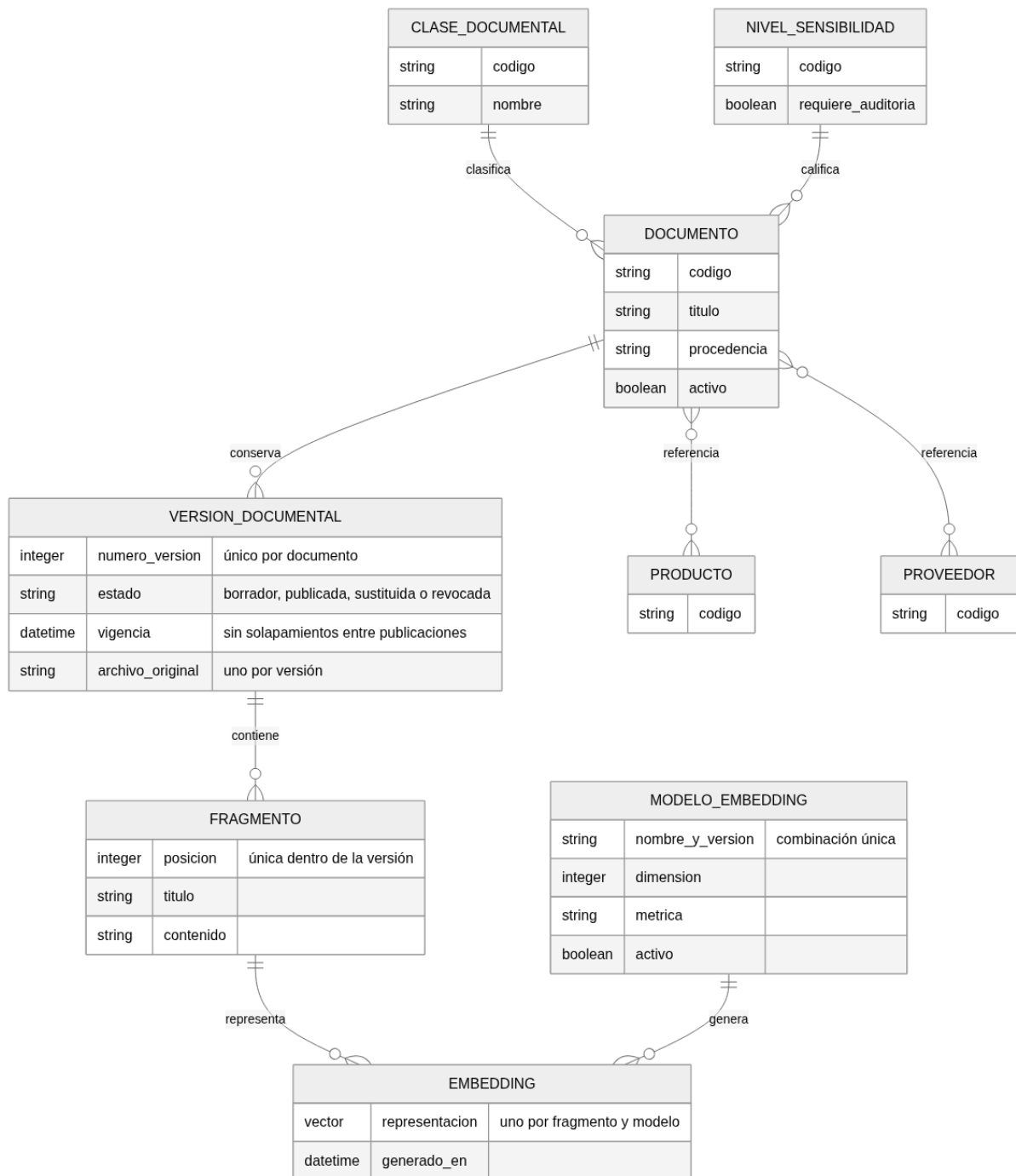


Figura 3: Vista conceptual de documentos y recuperación.

4.3. Acceso, interacción y evidencia

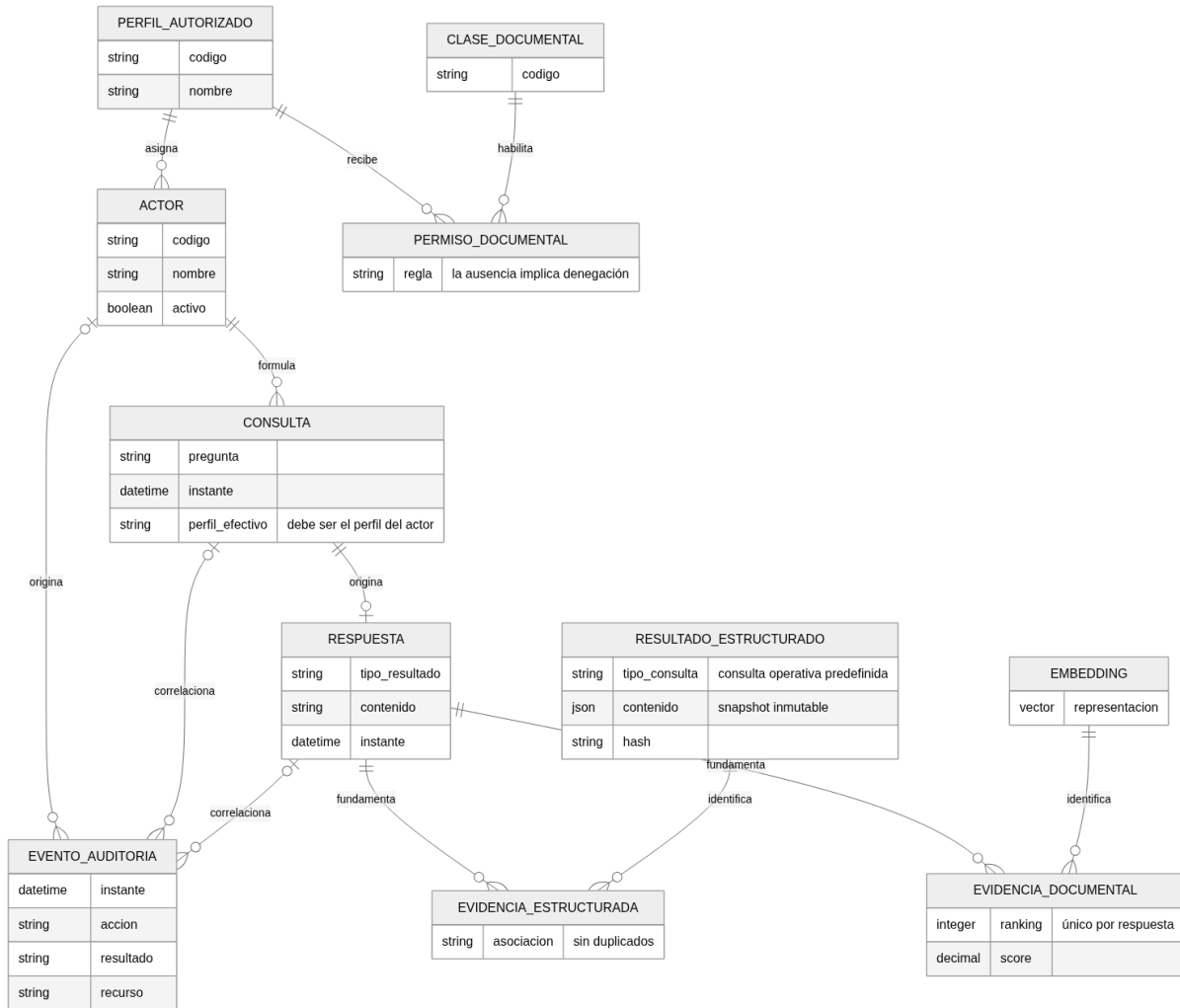


Figura 4: Vista conceptual de acceso, interacción y evidencia.

La Figura 4 vincula actores, perfiles, permisos, consultas, respuestas y evidencia. Cada actor tiene un perfil, y el permiso se define por clase documental. Una respuesta exitosa debe conservar evidencia documental o estructurada; un resultado negativo no admite evidencia. La auditoría registra el actor, el perfil efectivo y la correlación necesaria para reconstruir la interacción.

5. Modelo de implementación según la tecnología elegida

5.1. Modelo lógico relacional

El modelo lógico conserva las tres áreas conceptuales. Las tablas, columnas, claves y relaciones se reparten entre tres figuras. Las Figuras 5 a 7 deben leerse en conjunto.

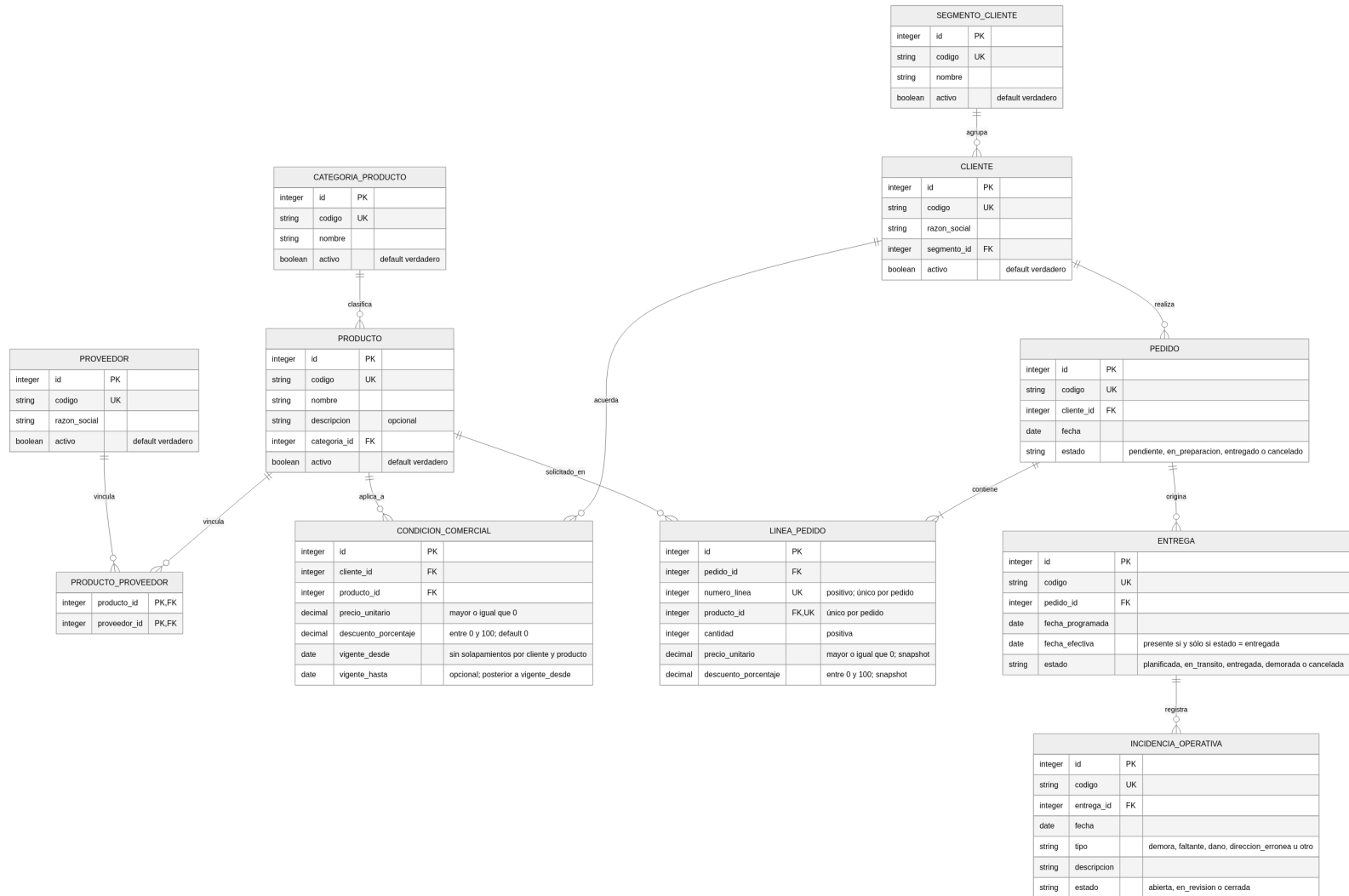


Figura 5: Modelo lógico de la operación comercial y logística.

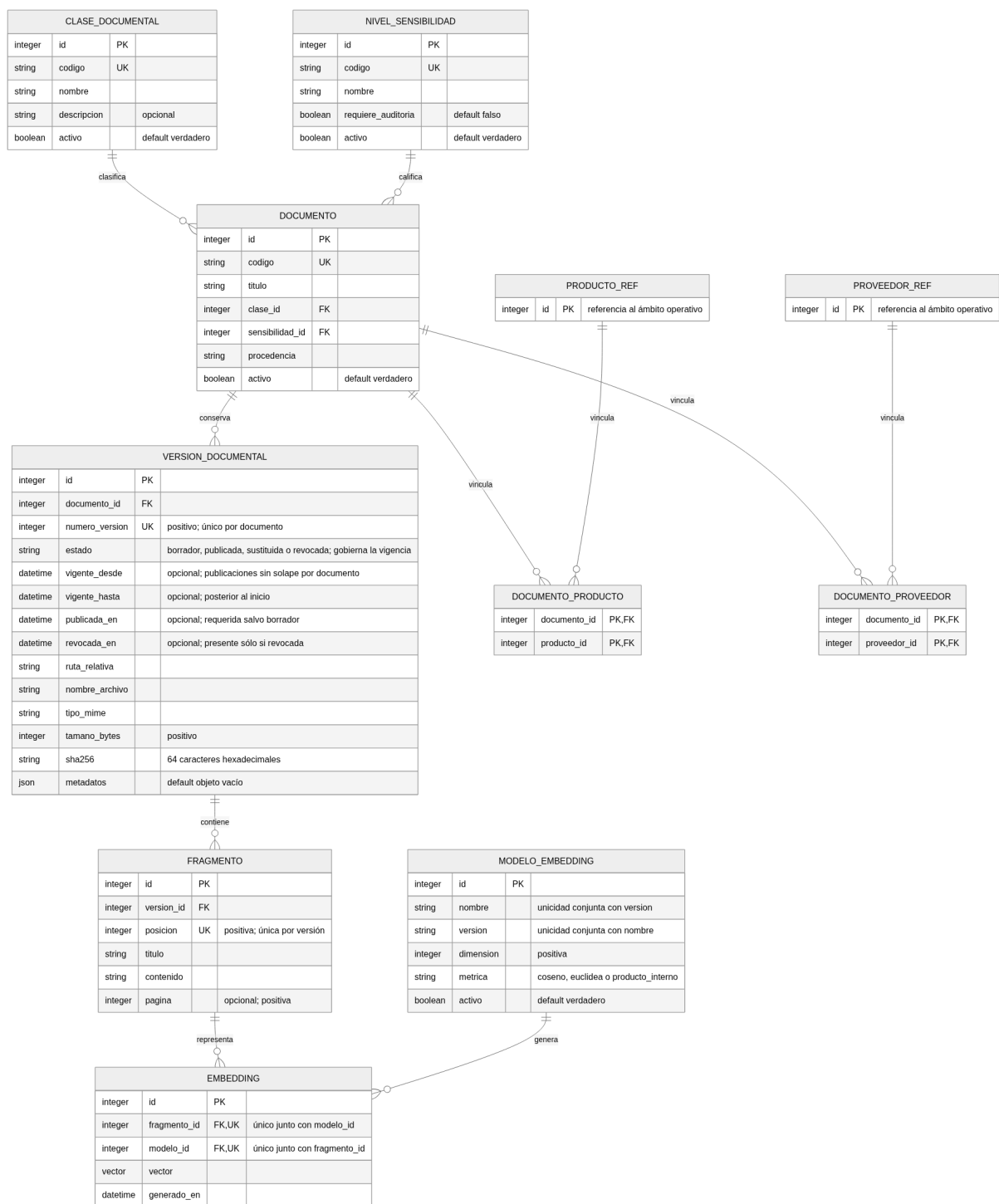


Figura 6: Modelo lógico de documentos y recuperación.

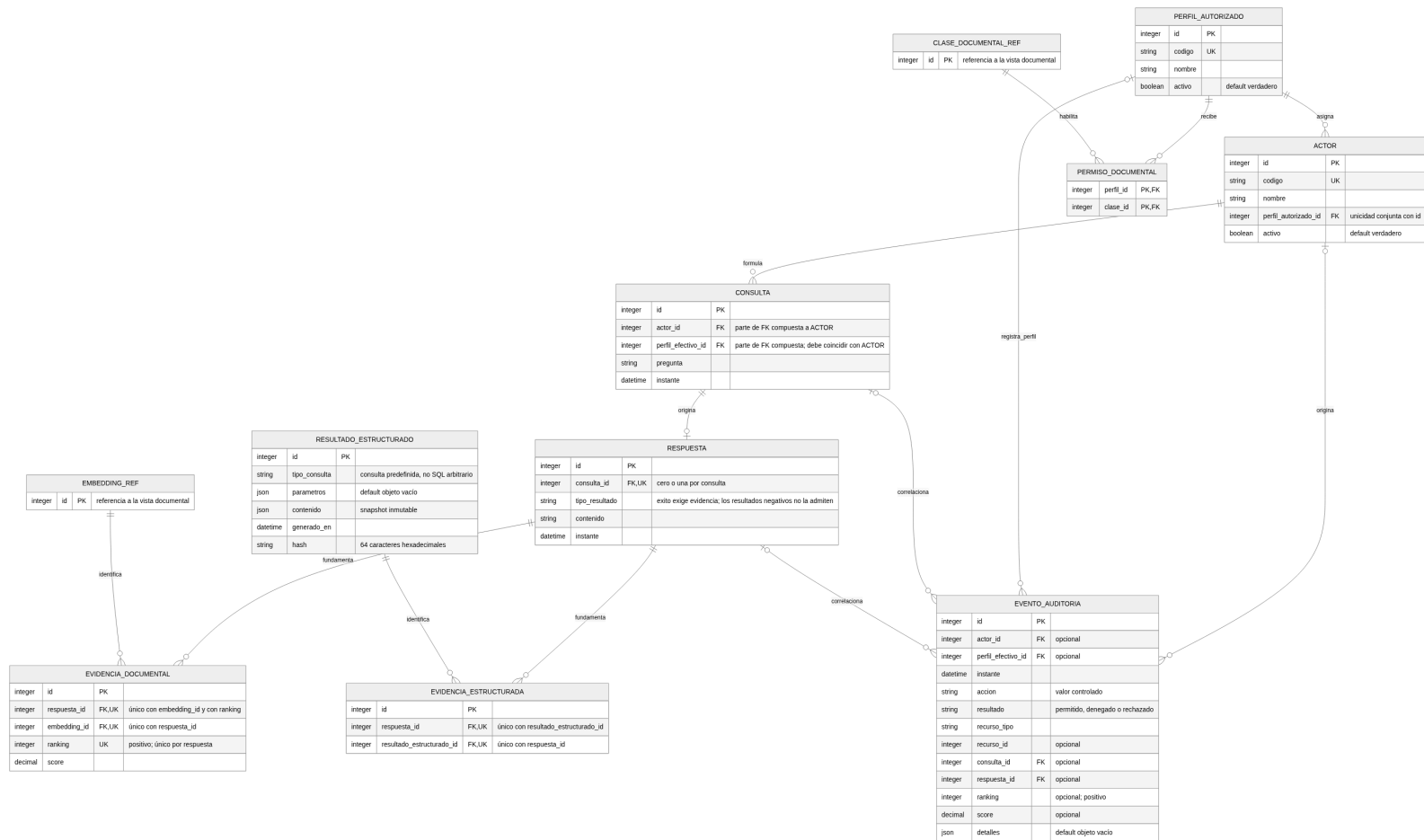


Figura 7: Modelo lógico de acceso, interacción y evidencia.

Las entidades con identidad propia usan claves sustitutas. Los códigos de producto, cliente, proveedor, pedido y documento son claves de negocio únicas. Las tablas asociativas usan claves compuestas cuando dos referencias identifican la fila. El recorrido documental enlaza `documento`, `version_documental`, `fragmento` y `embedding`. La evidencia conserva la referencia exacta de la fuente.

5.2. Modelo físico en PostgreSQL

El modelo físico lleva el diseño a PostgreSQL. Las Figuras 8 a 10 muestran las tablas, los tipos y las restricciones de cada área. Los scripts de `db/` definen el comportamiento implementado.

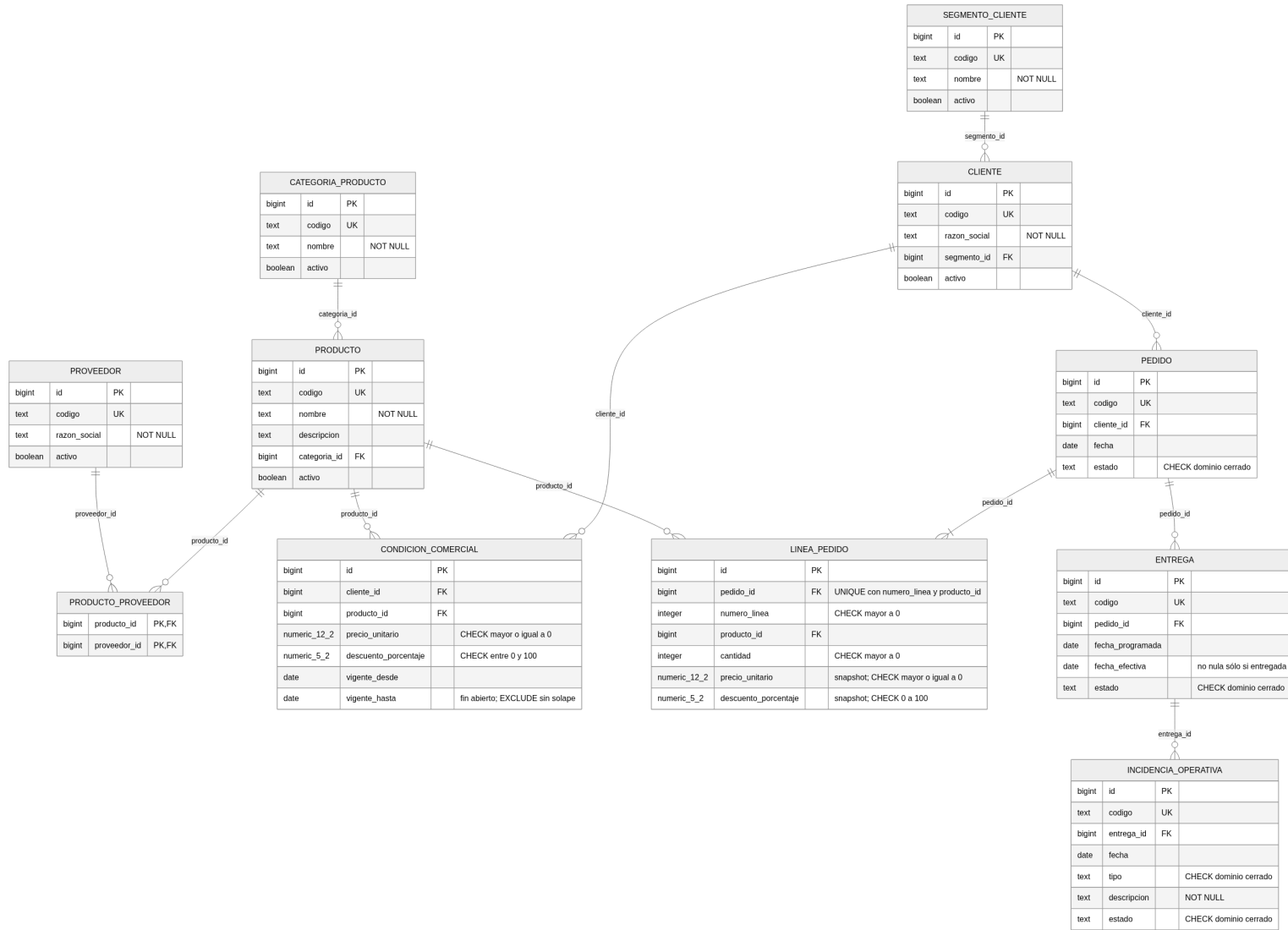


Figura 8: Implementación física de la operación comercial y logística.

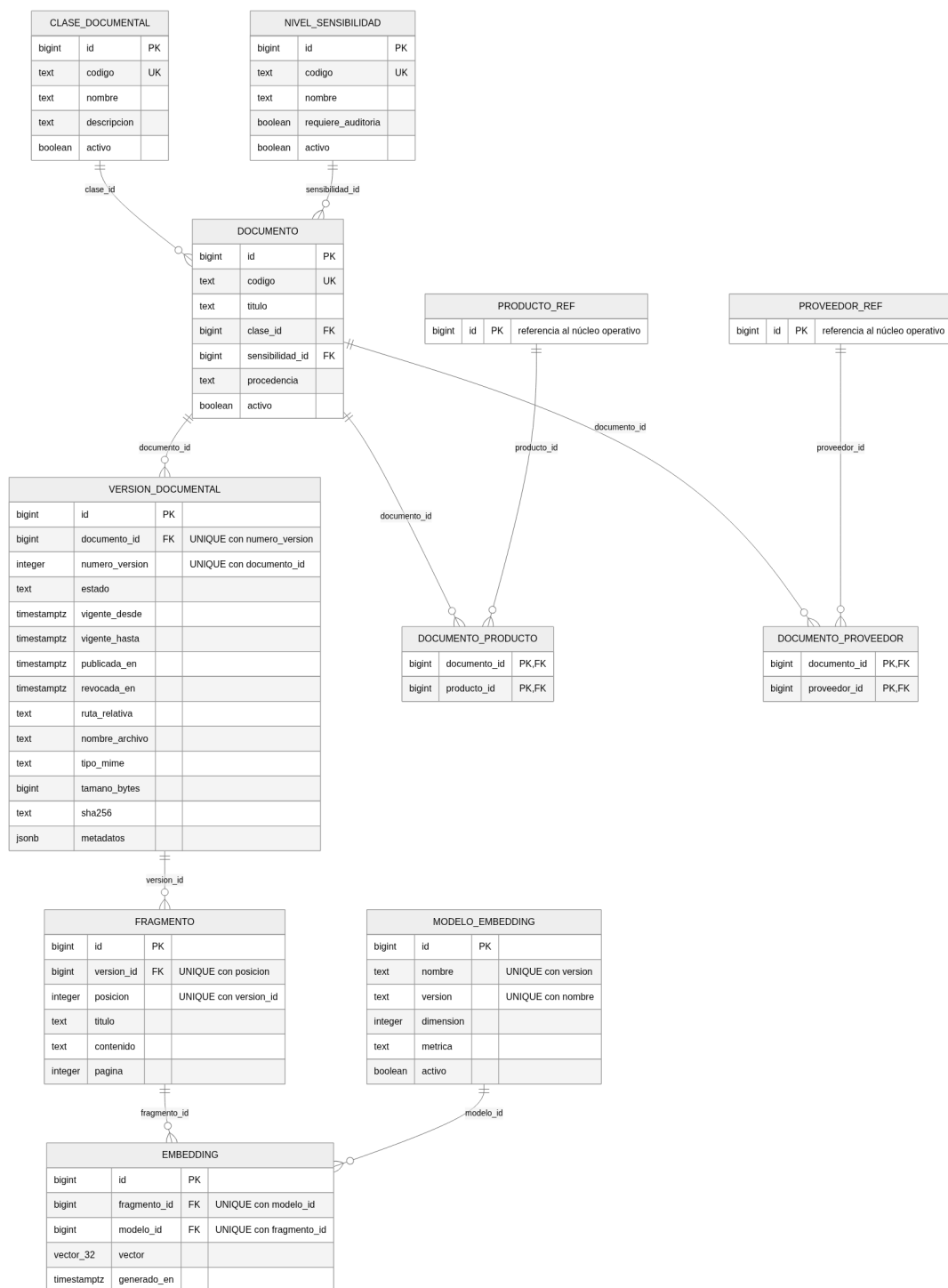


Figura 9: Almacenamiento, vigencia y recuperación vectorial en PostgreSQL.

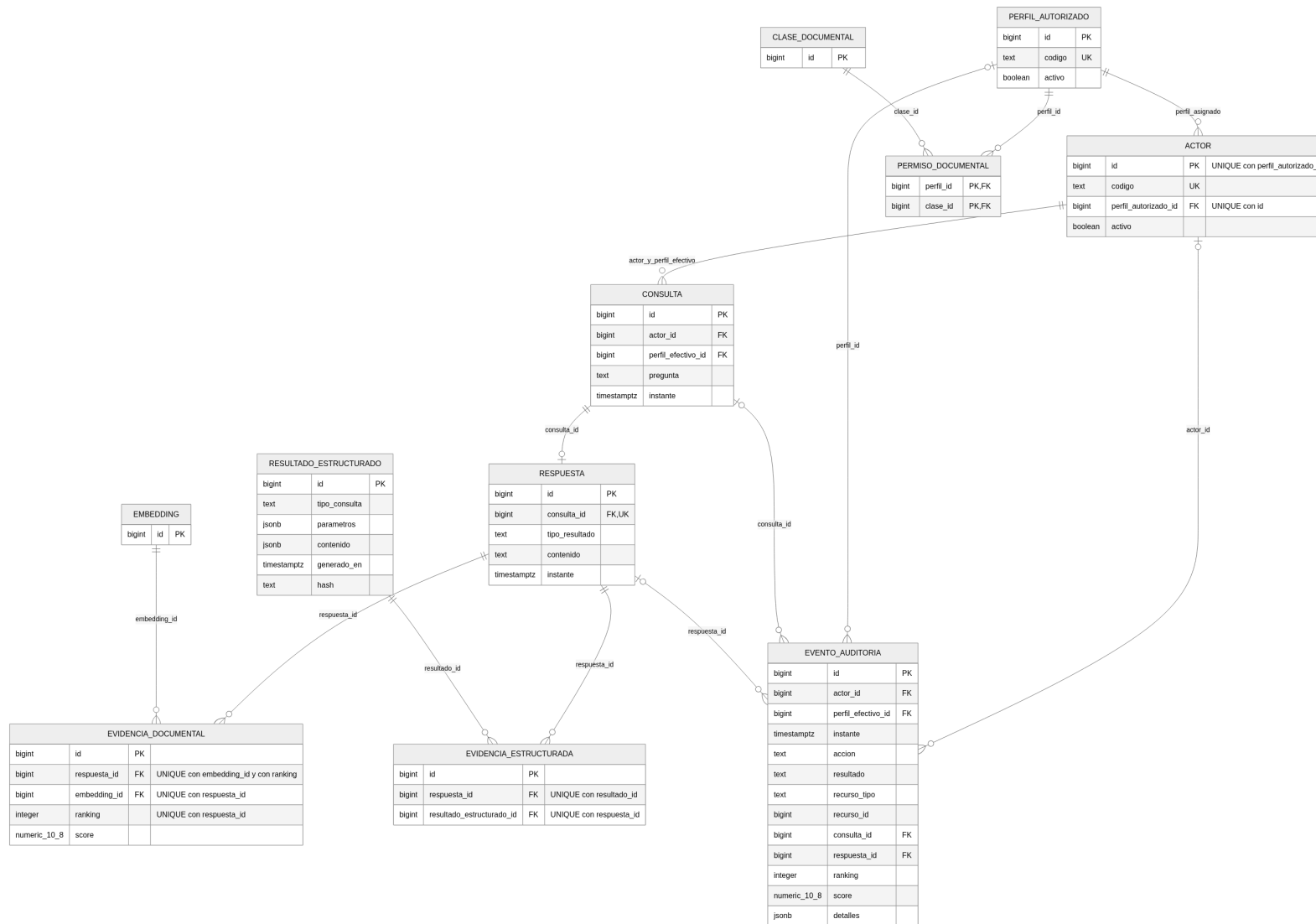


Figura 10: Autorización, evidencia y auditoría en PostgreSQL.

Decisión	Implementación	Propósito
Identidad y tiempo	<code>BIGINT IDENTITY</code> y <code>TIMESTAMPZ</code>	Identificar filas y comparar vigencias sin depender de la zona horaria.
Metadatos variables	<code>JSONB</code>	Conservar atributos heterogéneos sin trasladar el núcleo relacional a un modelo documental.
Vectores	<code>VECTOR(32)</code> e índice HNSW con distancia coseno	Ejecutar la prueba de recuperación top-k con vectores sintéticos y reproducibles.
Integridad	<code>CHECK</code> , claves foráneas, <code>UNIQUE</code> , <code>EXCLUDE</code> y triggers diferidos	Controlar estados, referencias, vigencias sin solape y reglas que abarcan varias filas.
Acceso y trazabilidad	Roles, privilegios, RLS y triggers de auditoría	Aplicar permisos antes del ranking y registrar operaciones sin permitir que el rol de ejecución altere el historial.

Cuadro 3: Principales decisiones del modelo físico.

Los índices B-tree cubren claves foráneas, filtros y joins usados por las consultas. HNSW cubre el acceso vectorial. Los intervalos semiabiertos y `EXCLUDE USING gist` impiden solapamientos en condiciones comerciales y versiones publicadas. Los constraint triggers diferidos validan al cerrar la transacción las reglas que abarcan varias filas.

6. Decisiones de normalización, embebido, referencia y desnormalización

La información que se identifica, relaciona, autoriza o filtra se mantiene tipada y normalizada. Categorías, perfiles, clases documentales y modelos de embedding se separan de las entidades que los usan. Las relaciones N:M usan tablas intermedias. Así no se repiten nombres ni reglas, y las claves foráneas rechazan referencias inválidas.

La desnormalización se reserva para hechos históricos. Cada línea de pedido guarda el precio y el descuento aplicados, aunque cambie la condición comercial. Un resultado estructurado conserva parámetros, contenido, instante y hash; la evidencia documental fija el embedding usado. Estas instantáneas no reemplazan a las entidades vigentes: guardan el estado necesario para explicar una operación o respuesta pasada.

`JSONB` se usa en cuatro lugares de estructura variable acotada: `version_documental.metadata`, `resultado_estructurado.parametros`, `resultado_estructurado.contenido` y `evento_auditoria.detalles`. No se crearon índices GIN porque los nueve casos verificables no filtran por claves internas de esos objetos. Las columnas que intervienen en integridad, autorización o joins permanecen fuera de `JSONB`.

Los archivos Markdown no se guardan en la base. Cada versión conserva una ruta relativa, nombre, tipo MIME, tamaño y SHA-256. Así PostgreSQL controla la identidad del archivo sin guardar binarios en la tabla de versiones.

7. Justificación de la tecnología seleccionada

Se usa PostgreSQL con `pgvector` y `btree_gist`. El caso requiere relaciones, vigencia temporal, autorización por fila, agregaciones SQL y búsqueda vectorial. Un solo motor aplica esas reglas en la misma transacción y evita sincronizar permisos y estados con un índice externo.

`pgvector` aporta `VECTOR(32)`, distancia coseno y el índice HNSW. `btree_gist` combina igualdad e intervalos en restricciones de exclusión. PostgreSQL también ofrece transacciones,

Alternativa	Ventaja posible	Motivo para no adoptarla como motor principal
Base documental	Flexibilidad para versiones y metadatos JSON.	Traslada relaciones, exclusión temporal y autorización uniforme a lógica de aplicación.
Clave-valor	Baja latencia para caché o contexto efímero.	No resuelve joins, agregaciones ni trazabilidad histórica como almacenamiento primario.
Columnar	Escritura sostenida de eventos y consultas por rango temporal.	La muestra necesita correlacionar auditoría con actores, consultas y respuestas mediante joins.
Grafo	Recorridos de relaciones variables.	Las relaciones del caso tienen profundidad conocida y se expresan con claves foráneas.
Vectorial dedicada	Mayor volumen de búsquedas por segundo a gran escala.	Exige replicar vigencia y permisos para filtrar antes del top-k, con riesgo de inconsistencia entre motores.

Cuadro 4: Alternativas consideradas frente a PostgreSQL.

claves foráneas, constraint triggers, roles y RLS. Estas funciones son centrales para el caso, no accesorios de la búsqueda semántica.

8. Implementación mínima realizada

La prueba funcional se ejecuta en fases numeradas. El esquema crea extensiones, tablas y restricciones. El seed carga los datos sintéticos. Los scripts siguientes agregan índices, la vista de recuperación, roles, privilegios, RLS y auditoría. Otros archivos ejecutan los casos C1–C9 y las validaciones negativas de integridad y seguridad.

`scripts/cargar_base.sh` regenera los artefactos, recrea la base y muestra los casos. `scripts/validar_base.sh` hace la validación completa. Sólo acepta una base terminada en `_validacion`; la recrea dos veces, comprueba conteos y checksums, prueba el rechazo de operaciones inválidas, valida permisos y compara instantáneas ordenadas.

La implementación termina en la capa de datos. No se agrega un backend ni una interfaz para simular una aplicación completa. Las pruebas usan SQL y roles técnicos para comprobar cada contrato de forma directa.

9. Datos de ejemplo utilizados

Los datos son sintéticos y reproducibles. El generador usa la semilla 42 y fija como referencia el 30 de junio de 2026 a las 15:00 UTC. El manifiesto guarda la versión del generador, los conteos, el inventario de archivos, sus checksums y los oráculos vectoriales.

Los escenarios se definieron antes de variar valores con la semilla. Incluyen un cliente sin pedidos, un pedido cancelado, entregas parciales, una demora con incidencia, condiciones vencidas y futuras, todos los estados documentales, un actor inactivo y accesos permitidos y denegados. Los 14 archivos Markdown son ficticios; cada encabezado de segundo nivel produce un fragmento. Los oráculos fijan como primeros resultados el fragmento 10 para la búsqueda vectorial y el 18 para la recuperación híbrida de Comercial/Compras.

Grupo	Cantidad	Componentes principales
Operación	12 productos, 12 pedidos	6 clientes, 28 líneas, 14 entregas y 4 incidencias
Documentación	10 documentos	14 versiones y 32 fragmentos
Vectores	36 embeddings	32 del modelo activo y 4 históricos
Acceso	6 actores	3 perfiles, 5 clases y 8 permisos
Interacción	7 pares	7 consultas, 7 respuestas y sus evidencias
Auditoría	56 eventos	Cambios documentales, consultas, accesos y evidencia

Cuadro 5: Escala principal del conjunto sintético.

10. Consultas representativas

El conjunto contiene siete consultas funcionales y dos pruebas transversales. Cada caso define sus parámetros, el resultado esperado y un orden estable con códigos de negocio.

Caso	Pregunta o control	Patrón y resultado comprobado
C1	Condición comercial vigente.	Filtra un intervalo semiabierto por cliente, producto y fecha; devuelve una fila aplicable.
C2	Pedidos y entregas que requieren atención.	Combina cuatro entidades con <code>JOIN/LEFT JOIN</code> ; incluye demoras y parciales, y excluye el pedido cancelado.
C3	Importe neto por categoría.	Usa <code>GROUP BY</code> sobre cantidad, precio y descuento; devuelve totales y conteos exactos.
C4	Productos principales por segmento.	Aplica <code>DENSE_RANK</code> ; el código de producto sólo estabiliza la presentación de empates.
C5	Clientes sin pedidos históricos.	Usa <code>NOT EXISTS</code> ; devuelve el único cliente reservado para ese escenario.
C6	Búsqueda vectorial top-k.	Ordena por distancia coseno y fragmento; el fragmento 10 ocupa la primera posición.
C7	Recuperación híbrida autorizada.	Combina similitud, vigencia, modelo activo y RLS; Comercial/Compras obtiene primero el fragmento 18 y Operaciones no ve su clase.
C8	Matriz de autorización.	Recorre las 15 combinaciones perfil-clase y confirma siete denegaciones por ausencia de permiso.
C9	Trazabilidad e inmutabilidad.	Correlaciona actor, consulta, respuesta, evidencia y auditoría; las modificaciones operativas de eventos son rechazadas.

Cuadro 6: Siete consultas y dos pruebas transversales.

C1–C7 producen los siete pares **consulta–respuesta**. C8 y C9 reutilizan esas interacciones para comprobar aislamiento y trazabilidad; no crean otras conversaciones. Por eso hay nueve casos SQL y siete interacciones simuladas.

11. Propuesta para datos semiestructurados, no estructurados y vectoriales

Cada fragmento documental es una unidad vectorizada. **embedding** referencia al fragmento y al modelo. La versión, el documento, la vigencia y la clase se obtienen mediante relaciones. La vista **fragmento_recuperable** reúne esos datos sin copiarlos en el vector, y RLS aplica los permisos del perfil efectivo sobre las tablas base.

Los vectores sintéticos tienen 32 dimensiones, están normalizados y forman vecindarios temáticos deterministas. La búsqueda usa distancia coseno ($\leq$), ordena de menor a mayor y desempata por **fragmento_id**. El modelo activo cubre todos los fragmentos. Cuatro también conservan embeddings históricos, excluidos de la recuperación vigente.

El índice HNSW usa `vector_cosine_ops`. La muestra es demasiado pequeña para evaluar rendimiento: PostgreSQL elige una exploración secuencial (`Seq Scan`) porque recorrer 32 vectores activos cuesta menos que usar el índice. Una ejecución diagnóstica con `enable_seqscan = off` confirma que HNSW puede resolver el patrón top-k, pero ese plan no se fuerza.

La recuperación debe evitar cuatro errores: usar versiones obsoletas, exponer clases no autorizadas, mezclar modelos y responder sin fuente. La vista filtra actividad, publicación, vigencia y modelo. RLS aplica el permiso antes del `ORDER BY` y del `LIMIT`; la unicidad evita duplicados; y la evidencia guarda el `embedding_id` exacto. Así una respuesta histórica sigue siendo explicable aunque cambie la versión vigente.

12. Propuesta de arquitectura de datos

La arquitectura usa una instancia de PostgreSQL para datos operativos, metadatos documentales, embeddings, permisos, interacciones y auditoría. Las extensiones `pgvector` y `btree_gist` agregan búsqueda vectorial y restricciones temporales sin sumar otro motor.

Un solo motor combina relaciones, vigencia, autorización y similitud en la misma consulta. Separar la base relacional de la vectorial obligaría a duplicar y sincronizar metadatos, sin una ventaja comprobable para esta escala.

La Figura 11 resume el recorrido. El generador produce el SQL de datos, los documentos Markdown y un manifiesto con conteos, checksums y oráculos. El cargador aplica el esquema, los datos, los índices, las vistas y la seguridad. Una aplicación futura podrá usar el rol de ejecución. El frontend, la API y la integración con un LLM quedan fuera de la implementación mínima.

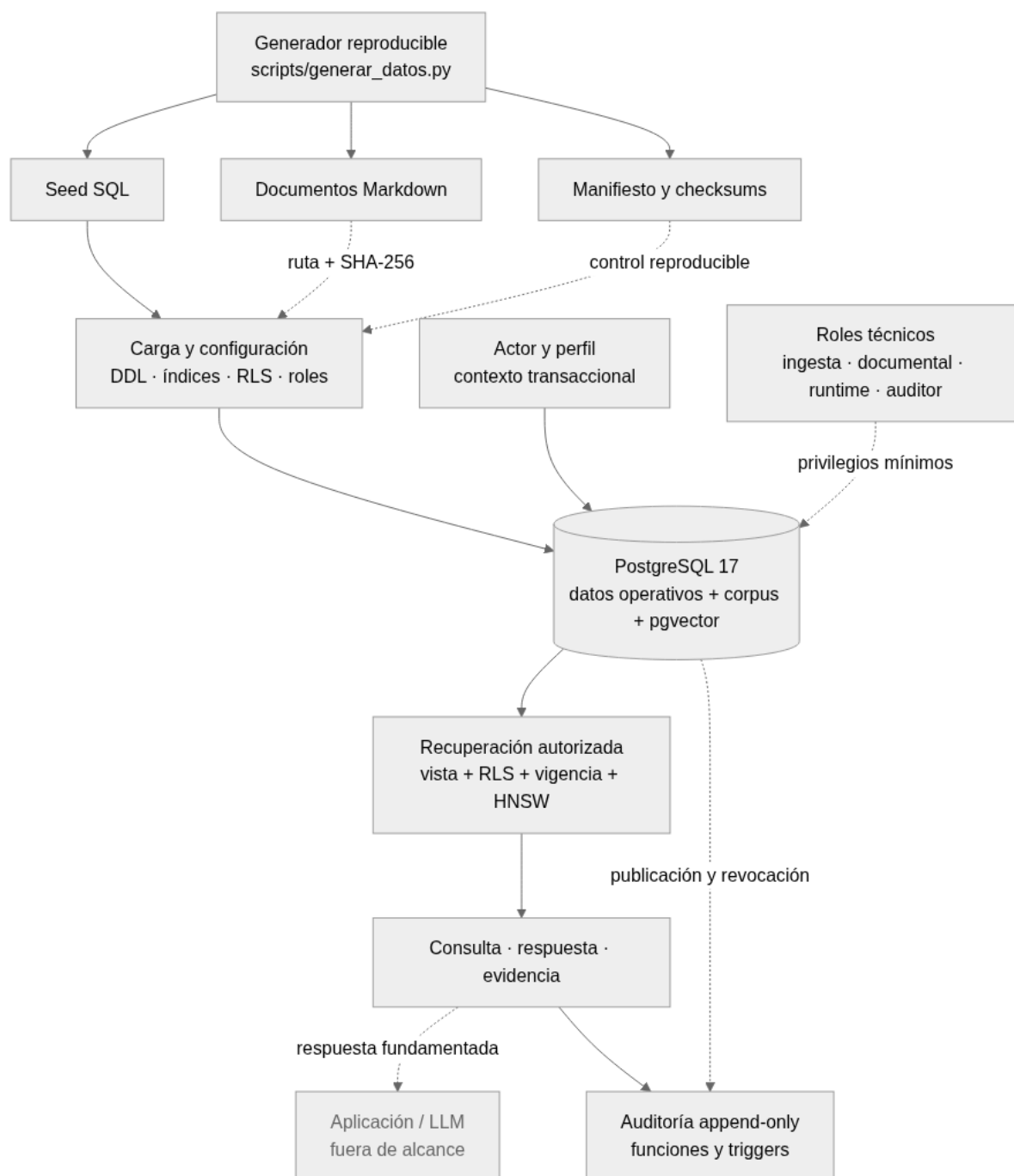


Figura 11: Arquitectura general y recorrido de los datos.

Los documentos quedan fuera de PostgreSQL. La base guarda su ruta relativa, procedencia, tamaño, tipo MIME y checksum. Así pueden trasladarse más adelante a almacenamiento de objetos sin cambiar el control de integridad.

No se incorpora un Data Warehouse, Data Lake ni Lakehouse porque las consultas son operativas y los datos son pocos. Si se necesitaran análisis históricos a otra escala, los hechos y eventos podrían copiarse a una capa analítica.

13. Estrategia de seguridad, permisos y aislamiento

Los perfiles funcionales son Operaciones y Logística, Comercial y Compras, y Administración y Calidad. Cada actor pertenece a un solo perfil. El permiso relaciona perfil y clase documental; si la combinación no existe, el acceso se deniega. El nivel de sensibilidad describe el dato, pero no reemplaza esta matriz.

Los roles técnicos separan tareas. `rag_ingesta` incorpora borradores y vectores; `rag_documental` publica o revoca mediante funciones atómicas; `rag_runtime` consulta y registra interacciones; y `rag_auditor` reconstruye evidencia en modo lectura. Todos son NOLOGIN, sin privilegios de superusuario ni BYPASSRLS. La autenticación queda para una aplicación futura y fuera del TP.

Cada transacción fija el código del actor en `rag.actor`. Las funciones de contexto comprueban que exista, esté activo y tenga un perfil activo. Un valor ausente, vacío, inexistente o inactivo produce un contexto nulo y las políticas deniegan la lectura. La base deriva el perfil desde el actor para que los valores no se contradigan. No autentica quién fija `rag.actor`: una aplicación futura deberá asociar la identidad autenticada con el actor de la transacción.

RLS se aplica y se fuerza sobre `documento`, `version_documental`, `fragmento` y `embedding`. También limita condiciones comerciales e incidencias según el perfil. La vista de recuperación usa los privilegios del invocador, no los del propietario. Las 15 combinaciones perfil-clase, los cuatro contextos inválidos y el acceso directo a tablas se prueban desde un rol no propietario.

La auditoría es append-only para los roles operativos: no pueden insertar eventos arbitrarios, modificarlos ni eliminarlos. Los triggers generan eventos para consultas, respuestas y evidencias; las funciones documentales registran publicación, sustitución y revocación dentro de la misma transacción. El propietario conserva TRUNCATE para recrear el conjunto sintético, una operación administrativa que no se concede a los roles de aplicación.

14. Consideraciones de escalabilidad y rendimiento

La muestra sirve para validar el diseño, no para medir rendimiento. La tabla más grande tiene 56 filas y `embedding` tiene 36. Los tiempos y planes observados no se pueden extrapolar a un sistema real.

Las tablas que más crecerían son `evento_auditoria`, `consulta`, `respuesta`, `fragmento` y `embedding`. La auditoría suma una fila por cada acción y no se elimina. Por eso es la primera candidata para un particionado mensual o trimestral, según el volumen y la retención necesaria.

Los fragmentos y embeddings crecen con cada documento o versión. Un fragmento puede tener un embedding por modelo, lo que permite conservar vectores históricos. Durante una migración de modelo, el volumen aumenta de forma temporal. Una política de retención puede limitar ese crecimiento.

Las consultas críticas son las búsquedas top-k y las híbridas autorizadas. Estas últimas combinan distancia coseno, estado, vigencia y permisos. Se creó un índice HNSW con `vector_cosine_ops` para el crecimiento previsto. En la muestra, el optimizador elige una exploración secuencial (Seq Scan) porque recorrer 36 vectores es más barato. El plan está documentado en `evidencias/planes_ejecucion.md`.

Los índices B-tree cubren los filtros y relaciones de las consultas: condiciones comerciales, entregas, incidencias, pedidos, vigencia y auditoría. No se indexa cada columna, porque los índices ocupan espacio y encarecen las escrituras.

Primero se aumentarían los recursos de PostgreSQL y se revisarían los planes de ejecución. Las réplicas de lectura sólo se justifican si crece el tráfico y las mediciones muestran un límite. La publicación, los permisos y la auditoría deben quedar en la instancia principal para conservar la consistencia transaccional.

Más adelante podrían separarse algunos componentes: archivos en almacenamiento de objetos, auditoría particionada o un motor orientado a escritura masiva. Una base vectorial dedicada sólo se justifica si los embeddings y el volumen de búsquedas por segundo crecen varios órdenes de magnitud. Esa opción obliga a sincronizar permisos y vigencia entre motores, con más complejidad y riesgo de inconsistencia.

La decisión debe basarse en crecimiento por tabla, latencia del top-k, costo de auditoría y tolerancia a consistencia diferida. Hasta que esas mediciones muestren un límite, PostgreSQL conserva las garantías actuales con menos coordinación.

15. Conclusiones

Se implementó una capa de datos pequeña para un copiloto RAG interno. PostgreSQL guarda datos operativos, documentos versionados, embeddings, permisos, interacciones y auditoría. La vigencia y la autorización reducen el universo antes del ranking, y cada respuesta conserva la evidencia que la sostuvo.

La prueba funcional valida claves, restricciones, invariantes diferidas, publicación atómica, RLS, privilegios mínimos y auditoría append-only mediante nueve casos. La semilla 42 produce los mismos archivos, conteos y checksums; la validación recrea la base dos veces y compara sus instantáneas. Para 36 embeddings, una exploración secuencial (**Seq Scan**) cuesta menos que usar HNSW. El índice queda disponible para un corpus mayor.

No se implementan autenticación, API, LLM, embeddings reales ni infraestructura distribuida, y la muestra no se usa como benchmark. Antes de escalar se deben medir el crecimiento y los cuellos reales. Según el resultado, se podrá particionar la auditoría, agregar réplicas, mover archivos o evaluar un motor vectorial separado. Cualquier cambio deberá mantener la autorización antes del ranking y la trazabilidad histórica.

A. Reproducibilidad y material de apoyo

Las instrucciones del `README.md` reproducen la carga y la validación. Las salidas extensas quedan en el repositorio y no se copian en el informe.

Artefacto	Qué permite comprobar
Manifiesto JSON	Procedencia, semilla, conteos, checksums y oráculos vectoriales.
Validaciones SQL	Integridad, aislamiento, ciclo documental y auditoría.
Matriz de autorización	Perfiles, contextos inválidos, autorización y trazabilidad.
Planes de ejecución	Planes observados, HNSW e índices B-tree.
Matriz de cobertura	Relación entre consigna, informe, implementación y evidencia.

Cuadro 7: Evidencias reproducibles del repositorio.

Los archivos están en `data/ejemplos/manifiesto.json`, `db/consultas/07_validaciones.sql`, `evidencias/seguridad/matriz_autorizacion.md`, `evidencias/planes_ejecucion.md` y `docs/matriz_cobertura.md`.